PinProbe A1

Box Control SCPI Command Reference

Communication Commands / Device Control / Error Handling

Customer Integration Material | **SCPI / UART** | **2026-07-28**

> Document updated: `2026-07-28` This document is intended for customer-side host integration, device control, and maintenance reference.

Device Information

Item	Default Value	Description
Manufacturer	`GTS`	`*IDN?` field 1; can be changed with `SYSTem:IDN1`
Model	`PINPROBEA1`	`*IDN?` field 2; can be changed with `SYSTem:IDN2`
Serial Number	`20250626`	`*IDN?` field 3; can be changed with `SYSTem:IDN3`
IDN Firmware Version	`V0.0.9`	`*IDN?` field 4; can be changed with `SYSTem:IDN4`
Compiled Firmware Version	`v1.0.0+<build-id>`	`SYSTem:VERSion?` returned

> `*IDN?` returns the current SCPI IDN configuration: `Manufacturer,Model,Serial Number,IDN Firmware Version` The IDN fields are stored in device configuration and may differ from compiled firmware version.

Communication Conventions

Item	Description
Serial Baud Rate	`115200`
Command Terminator	`\r\n`
Normal Response	Single-line text
Batch Log Response	`READ:LOG:ALL?` may return multi-line text
OTA Data Block	Uses SCPI definite-length binary block; max payload per block is `128` bytes

> SCPI errors can come from parser error queue, command responses beginning with `ERR...` text, and device exception logs. Host software should process both command responses and logs; see Error and Exception Handling.

Basic Commands (IEEE 488.2)

Command	Parameter	Response	Description
`*CLS`		None	Clear status and error queue
`*IDN?`		`GTS,PINPROBEA1,20250626,V0.0.9`	Query device identification
`*RST`		None	Reset SCPI parser state
`*STB?`		Status byte	Query status byte
`*WAI`		None	Wait for command completion
`*OPC?`		`1`	Query operation-complete status

System Commands

Command	Parameter	Response	Description
`SYSTem:ERRor[:NEXT]?`		Error code and description	Query next error message
`SYSTem:ERRor:COUNt?`		Error count	Query error queue count
`SYSTem:VERSion?`		`v1.0.0+<build-id>`	Query compiled firmware version and build ID
`SYSTem:UPTime?`		Seconds	Query system uptime
`SYSTem:REBoot`		`OK`	Software reset after about 200 ms
`SYSTem:FLASH:ID?`		`OK ID=<JEDEC> SR1=<xx> SR2=<xx>` or `ERR ...`	Query W25Q128 Flash ID and status registers

Example

```
SYSTem:ERRor:NEXT?
SYSTem:ERRor:COUNt?
SYSTem:VERSion?
SYSTem:UPTime?
SYSTem:FLASH:ID?
SYSTem:REBoot
```

Device Identification Configuration Commands

Command	Parameter	Response	Description
`SYSTem:IDN1`	`<string>`	None	Configure manufacturer field and save to Flash
`SYSTem:IDN1?`		String	Query manufacturer field
`SYSTem:IDN2`	`<string>`	None	Configure model field and save to Flash
`SYSTem:IDN2?`		String	Query model field
`SYSTem:IDN3`	`<string>`	None	Configure serial number/date field and save to Flash
`SYSTem:IDN3?`		String	Query serial number/date field
`SYSTem:IDN4`	`<string>`	None	Configure IDN firmware version field and save to Flash
`SYSTem:IDN4?`		String	Query IDN Firmware VersionField

Example

```
SYSTem:IDN1 "GTS"
SYSTem:IDN2 "PINPROBEA1"
SYSTem:IDN3 "20260727"
SYSTem:IDN4 "V0.0.9"
*IDN?
```

Communication Configuration Commands

Command	Parameter	Response	Description
`CONFigure:BAUDrate`	`115200`	`115200 baudrate is enable`	Configure BSM communication baud rate. Only `115200`

Example

```
CONFigure:BAUDrate 115200
```

Door and USB Control Commands

Command	Parameter	Response	Description
`CONFigure:CYLInder1`	`OPEN` / `CLOSE`	`OPEN` / `CLOSE`	Open or close the box door
`READ:CYLInder1:STATe?`		Actuator State	Query the current door actuator state
`CONFigure:CYLInder2`	`OPEN` / `CLOSE`	`OPEN` / `CLOSE`	Manual USB: `CLOSE` =insert/connect; `OPEN` =unplug/retract
`READ:CYLInder2:STATe?`		Actuator State	Query the current USB actuator state
`CONFigure:USB:AUTO`	`OFF` / `ON`	`OFF` / `ON`	Factory USB auto setting; **do not modify**
`READ:USB:AUTO?`		`OFF` / `ON`	Query the factory USB auto setting

> USB state naming: `CYLInder2 CLOSE` =insert/connect; `OPEN` =unplug/retract.

Actuator State

Return Value	Description
`CLOSE`	Command accepted; USB inserted/connected
`OPEN`	Command accepted; USB unplugged/retracted
`CLOSING`	Closing; USB insertion in progress
`OPENING`	Opening; USB retraction in progress
`CLOSED`	Closed; USB inserted/connected
`OPENED`	Opened; USB unplugged/retracted
`CYL ERR`	Actuator error

USB Automatic Sequence

`CONFigure:USB:AUTO ON` links USB insertion/retraction to door operation.

Scenario	Automatic Action
`READY`; both buttons confirmed 500 ms	Insert USB first; close after insertion is confirmed
USB insertion not reached or sensor fault	Do not close; log `USB_INSERT_FAIL`; retract USB; fast-flash red
`COMPLETE`; open-door button pressed	Open door; after `IDLE`, retract USB if still inserted
USB retraction not reached or output fault	Log `USB_RETRACT_FAIL`; fast-flash red; return `IDLE` if needed
Diagnostic fallback	If USB is not inserted after closing, log `USB_INSERT_FAIL`; flash yellow; restore green when ready

> `CONFigure:USB:AUTO` is factory-set; **do not modify**. Devices with a USB cylinder use `ON`; others use `OFF`. Confirm with `READ:USB:AUTO?`.

Example

```
CONFigure:CYLInder1 OPEN
CONFigure:CYLInder1 CLOSE
READ:CYLInder1:STATe?
CONFigure:CYLInder2 CLOSE
CONFigure:CYLInder2 OPEN
READ:CYLInder2:STATe?
READ:USB:AUTO?
```

Lock and Indicator Commands

Command	Parameter	Response	Description
CONFigure:LOCK	UNLOCK / LOCKED	UNLOCK / LOCKED	Set the device lock state
READ:LOCK:STATe?		Lock State	Read the current lock state

Lock State

Parameter / Return Value	Description
UNLOCK	Unlocked
LOCKED	Locked
LOCK ERR	Lock error

Example

```
CONFigure:LOCK LOCKED
CONFigure:LOCK UNLOCK
READ:LOCK:STATe?
```

LED Indicator Commands

Command	Parameter	Response	Description
CONFigure:LED	OFF / GREEN / RED / YELLOW	LED State	Set the LED output state
READ:LED:STATe?		LED State	Read the current LED output
CONFigure:LED:MAP	<io5>,<io6>,<io7>	G,R,Y format	Map IO5/IO6/IO7 to LED colors and save
READ:LED:MAP?		G,R,Y format	Read the IO5/IO6/IO7 LED color map

LED State

Parameter / Return Value	Description
OFF	LED off
GREEN	Green LED
RED	Red LED
YELLOW	Yellow LED
LED ERR	LED error

LED Mapping Parameters

Parameter	Description
G / GREEN	Map this IO line to the green LED
R / RED	Map this IO line to the red LED
Y / YELLOW	Map this IO line to the yellow LED

> CONFigure:LED:MAP needs three unique values and must include green, red, and yellow.

Example

```
CONFigure:LED GREEN
CONFigure:LED OFF
READ:LED:STATe?
CONFigure:LED:MAP G,R,Y
READ:LED:MAP?
```

System State Query Commands

Command	Parameter	Response	Description
`READ:SYSTem:STATe?`		System State	Query system state
`READ:IO:ALL?`		`IN:0xHH,0xHH OUT:0xHH,0xHH`	Query all raw input and output states

System StateReturn Value

Return Value	Description
`LOCK`	System locked
`IDLE`	System idle
`READY`	System ready
`RUNNING`	System running
`EMERGENCY`	Emergency stop triggered
`COMPLETE`	System operation complete
`SYS ERR`	System error

Example

```
READ:SYSTem:STATe?
READ:IO:ALL?
```

E-Stop, Risk Mode, and Boot Diagnostics

Command	Parameter	Response	Description
`CONFigure:ESTOP:TYPE`	`NC` / `NO`	`NC` / `NO`	Configure E-stop input type and save to Flash
`READ:ESTOP:TYPE?`		`NC` / `NO`	Query E-stop input type
`CONFigure:RISK:MODE`	`OFF` / `ON`	`OFF` / `ON`	Configure Risk Mode and save to Flash
`READ:RISK:MODE?`		`OFF` / `ON`	Query Risk Mode
`CONFigure:BOOT:DIAG`	`OFF` / `ON`	`OFF` / `ON`	Configure bootloader diagnostic serial output and save to Flash
`READ:BOOT:DIAG?`		`OFF` / `ON`	Query bootloader diagnostic serial output switch

ParameterDescription

Parameter	Description
`NC`	E-stop normally closed
`NO`	E-stop normally open
`OFF`	Disable function
`ON`	Enable function

Example

```
CONFigure:ESTOP:TYPE NC
READ:ESTOP:TYPE?
CONFigure:RISK:MODE ON
READ:RISK:MODE?
CONFigure:BOOT:DIAG ON
READ:BOOT:DIAG?
```

OTA Firmware Upgrade Commands

Command	Parameter	Response	Description
SYSTem:OTA:STATus?		STATE:... SIZE:... RECV:...	Query OTA receiving and verification status
SYSTem:OTA:BOOT?		FLASH:... FLAGS:... MAN:...	Query Bootloader and OTA Manifest status
SYSTem:OTA:BEGIN	<size>,<crc32>,<version>,<image_id>	OK or ERR,<code>,<name>	Start one OTA upload
SYSTem:OTA:DATA	<offset>,#<n><len><data>	OK,NEXT:<offset> or ERR,<code>,<name>	Write one OTA data block
SYSTem:OTA:END		OK or ERR,<code>,<name>	End OTA data receiving
SYSTem:OTA:VERify?		Status text or error text	Verify OTA image
SYSTem:OTA:COMMit		OK or error text	Commit upgrade request; device resets after success
SYSTem:OTA:ABORt		OK or error text	Abort current OTA process

OTA StatusResponse

SYSTem:OTA:STATus? response format:

```
STATE:<state> SIZE:<bytes> RECV:<bytes> BLOCKS:<recv>/<total> NEXT:<offset> CRC:0x<crc32> VER:0x<version> IMG:<id> ERR:<code>
```

SYSTem:OTA:BOOT? response format:

```
FLASH:<0|1> FLAGS:<0|1> MAN:<0|1> SEQ:<n> ACT:<n> PEND:<slot> PREV:<slot> BSTATE:<n> ATT:<n>/<max> BERR:<code> MSTATE:<state> SIZE:<bytes> CRC:0x<crc32> VER:0x<version> IMG:<id>
```

OTA Example

```
SYSTem:OTA:STATus?
SYSTem:OTA:BEGIN 4096,305419896,65536,1
SYSTem:OTA:DATA 0,#3128<128 bytes binary payload>
SYSTem:OTA:END
SYSTem:OTA:VERify?
SYSTem:OTA:COMMit
```

Error and Exception Handling

SCPI Parser Errors

When parameters are missing/invalid, a command is unknown, Flash save fails, or low-level hardware communication fails, firmware writes to the SCPI error queue. Queue capacity is `17` entries. Read with:

```
SYSTem:ERRor:COUNt?
SYSTem:ERRor:NEXT?
*CLS
```

Scenario	Typical Behavior	Recommended Handling
Unknown command	Serial port may output `**ERROR: -113, "Undefined header"`; queue may return `-113,"Undefined header"`	Stop sending this command; check spelling and hierarchy
Missing parameter	`-109,"Missing parameter"`	Provide required parameter
Illegal parameter value	`-224,"Illegal parameter value"` or `-222,"Data out of range"`	Check enum values, duplicate LED mapping, and numeric range
Flash save failure	`-320,"Storage fault"`	Configuration may not persist; retry and read back the related query command
UART/RS485 hardware or communication failure	`-240,"Hardware error"` or `-360,"Communication error"`	Query `READ:LOG:STATus?` / `READ:IO:ALL?`, and check IO expansion board link

> SCPI error queue is read by dequeue; `SYSTem:ERRor:NEXT?` pops one entry each time. `*CLS` clears status and error queue.

Command Responses Beginning with `ERR...`

Some commands return errors in normal responses instead of the SCPI error queue: `ERR...`:

Command	Error Format	Description
`SYSTem:FLASH:ID?`	`ERR INIT=<name>` or `ERR ID=<name>`	W25Q128 initialization or JEDEC ID read failed
OTA Command	`ERR,<code>,<name>`	OTA process error; see table below

OTA Error Codes

Code	Name	Meaning
`0`	`OK`	Success
`1`	`BUSY`	OTA process busy
`2`	`BAD_STATE`	Current state does not allow this OTA step
`3`	`BAD_PARAM`	Parameter error
`4`	`FLASH_FAIL`	Flash operation failed
`5`	`SIZE_OVERFLOW`	Image size exceeds allowed range
`6`	`CRC_FAIL`	CRC verification failed
`7`	`CAN_TIMEOUT`	CAN distribution timeout
`8`	`CAN_NACK`	CAN node rejected
`9`	`NODE_NOT_READY`	Node not ready
`10`	`BOOT_FAIL`	Bootloader upgrade failed
`11`	`APP_NOT_CONFIRMED`	App not confirmed
`12`	`BLOCK_MISSING`	OTA data block missing
`13`	`BAD_MANIFEST`	Invalid manifest
`14`	`UNSUPPORTED_IMAGE`	Unsupported image type

1. When command response begins with `ERR` or `ERROR` immediately read `SYSTem:ERRor:NEXT?` until no error remains.
2. Periodically read `READ:LOG:STATus?`; if `PEND` is nonzero, read `READ:LOG:NEXT?` or `READ:LOG:ALL?`.
3. When `[ERROR]` is found, stop automatic testing and collect `READ:SYSTem:STATe?` / `READ:IO:ALL?` / `READ:CYLInder1:STATe?` / `READ:CYLInder2:STATe?`.
4. When `[WARN]` is found, decide by event name; manually confirm USB failures, RS485 faults, and IO write failures before continuing.

> Recommended input air pressure: about `0.3 MPa`; set the electronic pressure gauge threshold to about `0.33 MPa`.

Exception	Log Level	Firmware Action	Recovery Recommendation
E-stop `ESTOP`	`[ERROR]`	Enter `EMERGENCY`, red light, open door, and lock system	After resetting E-stop and clearing laser path, press Start again to unlock and recover
Laser anti-pinch `LASER`	`[ERROR]`	If obstruction is detected late in closing, enter `EMERGENCY`, red light, open door, and lock system	Clear obstruction, then press Start again to unlock and recover
RS485 fault `RS485_FAULT`	`[WARN]`	Mark IO link unavailable; pause this operation and request yellow alarm	Wait for `RS485_RECOVERED`, then read IO and system state to confirm
Risk Mode pressure event `RISK_PRESSURE`	`[WARN]`	In Risk Mode, use pressure and learned time to determine door-close completion	Check door lower-limit or pressure sensor state
Low air pressure `AIR_LOW`	`[WARN]`	Record diagnostic log only; E-stop is not triggered	Wait for `clear_after` record or check air path
IO write failure `IO_WRITE_FAIL`	`[WARN]`	Related actuator command may not actually output	Check IO board communication, power, and actuator wiring
USB insertion failure `USB_INSERT_FAIL`	`[WARN]`	USB may retract, fast red flash, or yellow flash in `COMPLETE`	Query `READ:CYLInder2:STATe?`, check USB insertion cylinder and position sensor
USB retraction failure `USB_RETRACT_FAIL`	`[WARN]`	Fast red flashing; return to `IDLE`	Query `READ:CYLInder2:STATe?`, check USB retraction cylinder and position sensor

Debug Switch Commands

Command	Parameter	Response	Description
`CONFigure:DEBUg:STATe`	`OFF` / `ON`	`OFF` / `ON`	Enable/disable runtime state-transition debug output
`READ:DEBUg:STATe?`		`OFF` / `ON`	Query state-transition debug-output switch
`CONFigure:DEBUg:ACTion`	`OFF` / `ON`	`OFF` / `ON`	Enable/disable runtime action-duration debug output
`READ:DEBUg:ACTion?`		`OFF` / `ON`	Query action-duration debug-output switch
`CONFigure:DEBUg:EVENt`	`OFF` / `ON`	`OFF` / `ON`	Enable/disable runtime event debug output
`READ:DEBUg:EVENt?`		`OFF` / `ON`	Query event debug-output switch
`CONFigure:DEBUg:IO`	`OFF` / `ON`	`OFF` / `ON`	Enable/disable runtime IO debug output
`READ:DEBUg:IO?`		`OFF` / `ON`	Query IO debug-output switch

> Debug switches are runtime state; default is `OFF`, not written to Flash.

Example

```
CONFigure:DEBUg:STATe ON
READ:DEBUg:STATe?
CONFigure:DEBUg:IO OFF
READ:DEBUg:IO?
```

Device Log Commands

Command	Parameter	Response	Description
`CONFigure:LOG:UART`	`OFF` / `ON`	`OFF` / `ON`	Enable/disable live UART logging
`READ:LOG:UART?`		`OFF` / `ON`	Read live UART logging setting
`READ:LOG:NEXT?`		One log entry or `EMPTY`	Read the next pending device log
`READ:LOG:ALL?`		Multi-line logs or `EMPTY`	Read all pending device logs
`READ:LOG:STATus?`		`PEND:... CAP:... NEXT:...`	Read the log buffer status
`CONFigure:LOG:CLEar`		`OK`	Clear the log buffer and counters

Log Status Response

```
PEND:<pending> CAP:<capacity> NEXT:<next_seq> DROP:<drop_count> RDROP:<realtime_drop_count> UART:<0|1>
```

Log Line Format

Logs usually start with timestamp and node number:

```
[T+12.345s][N0][INFO][ACTION] CLOSE_DONE duration=300ms
```

Format:

```
[T+<time since boot>][N<node number>][<level>][<module>] <event and parameters>
```

Field	Description
`T+`	Boot-relative time; restarts after reset
`N`	Node number; single-device default is `N0`
`INFO`	Normal state, action, command, or recovery
`WARN`	Exception or diagnostic record to check
`ERROR`	Severe event, e.g. E-stop or laser anti-pinch
`STATE` / `ACTION` / `EVENT` / `IO`	State, action, event, or IO snapshot

Exception Log Description

Use the log level to judge exceptions. E-stop and laser anti-pinch use `[ERROR][EVENT]`; other diagnostics usually use `[WARN][EVENT]`:

Exception Event	Log Example	Meaning and Operation Notes
E-stop triggered	`[T+15.120s][N0][ERROR][EVENT] ESTOP close_elapsed=820ms`	Opens door and locks; reset E-stop, press Start
Object obstruction	`[T+16.020s][N0][ERROR][EVENT] LASER close_elapsed=1720ms`	Opens door and locks; clear obstruction, press Start
RS485 fault	`[T+12.345s][N0][WARN][EVENT] RS485_FAULT arg0=0 arg1=0`	IO link unavailable; wait for recovery log
Risk pressure	`[T+16.100s][N0][WARN][EVENT] RISK_PRESSURE close_elapsed=1800ms`	Close-pressure condition; check workpiece/resistance
Air low / recovered	`[T+20.000s][N0][WARN][EVENT] AIR_LOW low_after=500ms`	Low pressure or recovery; check `low_after/clear_after`
IO write fail	`[T+21.000s][N0][WARN][EVENT] IO_WRITE_FAIL target=2 cmd=0x0003`	Output write failed; check actuator and IO link
USB insert fail	`[T+22.000s][N0][WARN][EVENT] USB_INSERT_FAIL elapsed=2001ms reason=timeout`	USB not inserted; check `reason` cause
USB retract fail	`[T+23.000s][N0][WARN][EVENT] USB_RETRACT_FAIL elapsed=2001ms reason=sensor_conflict`	USB not retracted; check `reason` cause

USB Reasons

reason	Description
`timeout`	USB travel exceeded 2000 ms without reaching position
`sensor_conflict`	USB upper/lower sensors active at the same time
`dual_output`	USB insert/retract outputs active at the same time
`unknown`	Unrecognized reason code

A normal information log is generated when RS485 communication recovers:

```
[T+13.500s][N0][INFO][EVENT] RS485_RECOVERED arg0=0 arg1=0
```

This record means the IO link recovered; the device exits RS485 protection and requests alarm light off. `ESTOP` and `LASER` open the door and lock the system; clear the cause, then press Start. `RISK_PRESSURE` has no separate recovery event; judge from later system, IO, and action logs.

Reading Tips

1. Send `READ:LOG:STATus?` first to check pending and drop counts.
2. Use `READ:LOG:NEXT?` one by one, or `READ:LOG:ALL?` for all entries.
3. Filter `[WARN]` and `[ERROR]`; after `RS485_FAULT`, wait for `RS485_RECOVERED`.
4. If `DROP` or `RDROP` is nonzero, logs are incomplete; collect system and IO states.

> `READ:LOG:NEXT?` and `READ:LOG:ALL?` are dequeue reads; returned records are not repeated. The buffer holds `76` RAM entries and clears after reset or power loss. `DROP` counts overwritten pending records; `RDROP` counts records lost when real-time UART cannot send fast enough. `CONFigure:LOG:CLEar` clears logs and both drop counters. With `CONFigure:LOG:UART ON`, logs appear as unsolicited text on the SCPI port. Keep `OFF` if async text cannot be handled.

Example

```
CONFigure:LOG:UART ON
READ:LOG:UART?
READ:LOG:STATus?
READ:LOG:NEXT?
READ:LOG:ALL?
CONFigure:LOG:CLEar
```

Command Summary

Category	Command
Basic	`*CLS`, `*IDN?`, `*RST`, `*STB?`, `*WAI`, `*OPC?`
System	`SYSTem:ERRor[:NEXT]?`, `SYSTem:ERRor:COUNt?`, `SYSTem:VERSion?`, `SYSTem:UPTime?`, `SYSTem:REBoot`, `SYSTem:FLASH:ID?`
IDN	`SYSTem:IDN1`, `SYSTem:IDN1?`, `SYSTem:IDN2`, `SYSTem:IDN2?`, `SYSTem:IDN3`, `SYSTem:IDN3?`, `SYSTem:IDN4`, `SYSTem:IDN4?`
Act.	`CONFigure:CYLInder#`, `READ:CYLInder#:STATe?`, `CONFigure:USB:AUTO`, `READ:USB:AUTO?`, `CONFigure:LOCK`, `READ:LOCK:STATe?`, `CONFigure:LED`, `READ:LED:STATe?`, `CONFigure:LED:MAP`, `READ:LED:MAP?`
Status	`READ:SYSTem:STATe?`, `READ:IO:ALL?`
Cfg.	`CONFigure:BAUDrate`, `CONFigure:ESTOP:TYPE`, `READ:ESTOP:TYPE?`, `CONFigure:RISK:MODE`, `READ:RISK:MODE?`, `CONFigure:BOOT:DIAG`, `READ:BOOT:DIAG?`
OTA	`SYSTem:OTA:STATus?`, `SYSTem:OTA:BOOT?`, `SYSTem:OTA:BEGIN`, `SYSTem:OTA:DATA`, `SYSTem:OTA:END`, `SYSTem:OTA:VERify?`, `SYSTem:OTA:COMMit`, `SYSTem:OTA:ABORt`
Dbg.	`CONFigure:DEBUg:STATe`, `READ:DEBUg:STATe?`, `CONFigure:DEBUg:ACTion`, `READ:DEBUg:ACTion?`, `CONFigure:DEBUg:EVENT`, `READ:DEBUg:EVENt?`, `CONFigure:DEBUg:IO`, `READ:DEBUg:IO?`
Log	`CONFigure:LOG:UART`, `READ:LOG:UART?`, `READ:LOG:NEXT?`, `READ:LOG:ALL?`, `READ:LOG:STATus?`, `CONFigure:LOG:CLEar`

Revision History

Date	New Commands	Revision
`2026-07-28`	`CONFigure:CYLInder2` / `READ:CYLInder2:STATe?`	Revised USB manual control and state return semantics: `CLOSE/CLOSED` indicates USB inserted/connected; `OPEN/OPENED` indicates USB unplugged/retracted; hardware output behavior is unchanged.
`2026-07-27`	`CONFigure:USB:AUTO` / `READ:USB:AUTO?`	Added automatic USB plug/unplug sequence commands; factory configured by device hardware, **Do not modify**; devices with USB cylinder are `ON`, devices without USB cylinder are `OFF`; customer software uses `READ:USB:AUTO?` to confirm whether enabled.